

程序报告：斑马问题

学号： 2312631

姓名：林盛森

一、问题重述

这个问题就是给了 5 个待我们去填充的变量：units，对于每个 unit，其内部有 5 条属性，我们需要根据已有信息填入相应属性，并根据逻辑推理，推理出未匹配的属性应该处在哪个 unit 中。

斑马问题： 5 个不同国家（英国、西班牙、日本、意大利、挪威）且工作各不相同（油漆工、摄影师、外交官、小提琴家、医生）的人分别住在一条街上的 5 所房子里，每所房子的颜色不同（红色、白色、蓝色、黄色、绿色），每个人都有自己养的不同宠物（狗、蜗牛、斑马、马、狐狸），喜欢喝不同的饮料（矿泉水、牛奶、茶、橘子汁、咖啡）。根据以下提示，你能告诉我哪所房子里的人养斑马，哪所房子里的人喜欢喝矿泉水吗？

- 1.英国人住在红色的房子里
- 2.西班牙人养了一条狗
- 3.日本人是一个油漆工
- 4.意大利人喜欢喝茶
- 5.挪威人住在左边的第一个房子里
- 6.绿房子在白房子的右边
- 7.摄影师养了一只蜗牛
- 8.外交官住在黄房子里
- 9.中间那个房子的人喜欢喝牛奶
- 10.喜欢喝咖啡的人住在绿房子里
- 11.挪威人住在蓝色的房子旁边
- 12.小提琴家喜欢喝橘子汁
- 13.养狐狸的人所住的房子与医生的房子相邻
- 14.养马的人所住的房子与外交官的房子相邻

二、设计思想

1. 导入 python 包

```
from kanren import run, eq, membero, var, conde
from kanren.core import lall
import time
```

kanren 一个描述性Python逻辑编程系统
lall 包用于定义规则

导入 kanren 和 lall 包，kanren 为我们提供智能体从而对逻辑进行推理分析，lall 用于定义规则，方便我们为智能体提供有效信息；

2. 实现 left, next 逻辑

```
def left(x, y, units):
    next_units_in_order = zip(units[:4], units[1:])
    return membro((x, y), next_units_in_order) # (x,y) 是 (units[i],units[i+1]) 的子集

def next(x, y, units):
    return conde([left(x, y, units)], [left(y, x, units)]) # 【】表示逻辑或
```

由于在设定的 14 条信息中，有对应房子 unit 的位置信息，我们需要自行构建两个函数，left 函数说明第一个参数在第二个参数的左面，对于 right 逻辑我们只需更换参数的位置即可，在 left 函数内部，我们先对房子进行了一些处理，把相邻两个房子放在一个二元组里，依次为（房子 0，房子 1），（房子 1，房子 2），（房子 2，房子 3），（房子 3，房子 4），这样就实现了判断两个 unit 的位置关系的处理，对于返回值，membro 是 kanren 包中的函数，用于判断第一个参数是不是后一个参数容器的元素，这样，我们给入两个 unit 值，如果 (x, y) 是连续的，即 x 房子在 y 房子的左边，则说明我们需要添加 membro 这条逻辑提供给智能体；

而对于 next 函数，只需添加 x 在 y 的左边或 y 在 x 的左边的信息即可；

3. 智能体实现

```
self.units = [(var(),var(),var(),var(),var())for i in range(5)]
```

首先要对 units 初始化，units 是一个有 5 个 unit 元组的列表，每一个 unit 内部还有 5 条属性；

4. 添加 14 条逻辑信息

```
(membro, ('英国人', var(), var(), var(), '红色'), self.units),
(membro, ('西班牙人', var(), var(), '狗', var()), self.units),
(membro, ('日本人', '油漆工', var(), var(), var()), self.units),
(membro, ('意大利人', var(), '茶', var(), var()), self.units),
(eq, ('挪威人', var(), var(), var(), var()), self.units[0]),
(left, (var(), var(), var(), var(), '白色'), (var(), var(), var(), var(), '绿色'), self.units),
(membro, (var(), '摄影师', var(), '蜗牛', var()), self.units),
(membro, (var(), '外交官', var(), var(), '黄色'), self.units),
(eq, (var(), var(), '牛奶', var(), var()), self.units[2]),
(membro, (var(), var(), '咖啡', var(), '绿色'), self.units),
(next, ('挪威人', var(), var(), var(), var()), (var(), var(), var(), var(), '蓝色'), self.units),
(membro, (var(), '小提琴家', '橘子汁', var(), var()), self.units),
(next, (var(), var(), var(), '狐狸', var()), (var(), '医生', var(), var(), var()), self.units),
(next, (var(), var(), var(), '马', var()), (var(), '外交官', var(), var(), var()), self.units),
(membro, (var(), var(), '斑马', var()), self.units),
(membro, (var(), var(), '矿泉水', var(), var()), self.units)
```

接着我们需要把题干给出来的 14 条逻辑信息提供给智能体，membro 这个函数我们已经在 left 函数中说明，eq 这个函数是说明两个参数相等，left，next 我们也已经说明过；

5. 规则求解器

```
def solve(self):
    """
    规则求解器(请勿修改此函数).
    return: 斑马规则求解器给出的答案，共包含五条匹配信息，解唯一。
    """

    self.define_rules()
    self.solutions = run(0, self.units, self.rules_zebraproblem)
    return self.solutions
```

通过调用 run 函数，我们即可得到五条匹配信息。

三、代码内容

```
from kanren import run, eq, membero, var, conde          # kanren 一个描述性 Python 逻辑编程
系统
from kanren.core import lall                            # lall 包用于定义规则
import time

=====

def left(x, y, units):
    next_units_in_order = zip(units[:4], units[1:])
    return membero((x, y), next_units_in_order) # (x,y)是 (units[i],units[i+1]) 的子集

def next(x, y, units):
    return conde([left(x, y, units)], [left(y, x, units)]) # 【】表示逻辑或

=====

class Agent:
    """
    推理智能体.
    """

    def __init__(self):
        """
        智能体初始化.
        """

        self.units = [(var(),var(),var(),var(),var())for i in range(5)]
        self.rules_zebraproblem = None # 用 lall 包定义逻辑规则
        self.solutions = None          # 存储结果

=====

    def define_rules(self):
        """
        定义逻辑规则.
        """

        self.rules_zebraproblem = lall(
            (membero, ('英国人', var(), var(), var(), '红色'), self.units),
            (membero, ('西班牙人', var(), var(), '狗', var()), self.units),
            (membero, ('日本人', '油漆工', var(), var(), var()), self.units),
            (membero, ('意大利人', var(), '茶', var(), var()), self.units),
            (eq, ('挪威人', var(), var(), var(), var()), self.units[0]),
```

```

        (left, (var(), var(), var(), var(), '白色'),(var(), var(), var(), var(), '绿色'), self.units),
        (membero, (var(), '摄影师', var(), '蜗牛', var()), self.units),
        (membero, (var(), '外交官', var(), var(), '黄色'), self.units),
        (eq, (var(), var(), '牛奶', var(), var()), self.units[2]),
        (membero, (var(), var(), '咖啡', var(), '绿色'), self.units),
        (next, ('挪威人', var(), var(), var(), var()), (var(), var(), var(), var(), '蓝色'),
self.units),

        (membero, (var(), '小提琴家', '橘子汁', var(), var()), self.units),
        (next, (var(), var(), var(), '狐狸', var()), (var(), '医生', var(), var(), var()), self.units),
        (next, (var(), var(), var(), '马', var()), (var(), '外交官', var(), var(), var()), self.units),
        (membero,(var(), var(), var(), '斑马', var()), self.units),
        (membero,(var(), var(), '矿泉水', var(), var()), self.units)

    )
=====

def solve(self):
    """
    规则求解器(请勿修改此函数).
    return: 斑马规则求解器给出的答案，共包含五条匹配信息，解唯一.
    """

    self.define_rules()
    self.solutions = run(0, self.units, self.rules_zebraproblem)
    return self.solutions
=====

agent = Agent()
solutions = agent.solve()

# 提取解释器的输出
output = [house for house in solutions[0] if '斑马' in house][0][4]
print ("\n{} 房子里的人养斑马".format(output))
output = [house for house in solutions[0] if '矿泉水' in house][0][4]
print ('{} 房子里的人喜欢喝矿泉水'.format(output))

# 解释器的输出结果展示
for i in solutions[0]:
    print(i)
=====

```

四、实验结果

实验结果如图所示：

绿色房子里的人养斑马
黄色房子里的人喜欢喝矿泉水
('挪威人', '外交官', '矿泉水', '狐狸', '黄色')
('意大利人', '医生', '茶', '马', '蓝色')
('英国人', '摄影师', '牛奶', '蜗牛', '红色')
('西班牙人', '小提琴家', '橘子汁', '狗', '白色')
('日本人', '油漆工', '咖啡', '斑马', '绿色')

测试详情

测试点	状态	时长	结果
测试结果	✓	2s	测试成功!

确定

五、总结

1. 通过这次实验认识到了智能体的强大性，其实不通过这种智能体的逻辑算法来解决斑马问题，用穷举或者是人为地去判断的话，也可以实现，但需要耗费很长的时间，效率明显是智能体更高一些，还不容易出错；

2. 通过添加相应代码解决了斑马问题，提升了 python 编写函数的能力，了解了 kanren 这个 python 包，并且能够读懂 kanren 的语句和函数；

3. 实验时发现对 var() 变量无法通过下标对其访问，会出现“'Var' object is not subscriptable”的报错，而 14 条信息中有“在左面第一个房子”这样的信息，所以我把 units 改成了列表容器，从而可以直接利用下标 units[i] 进行访问是第几个房子；

4. 在写完 14 条逻辑信息后，运行程序，发现匹配结果里没有“斑马”和“矿泉水”的信息，原因是 14 条逻辑信息里没有这两个属性，我们需要自行添加进去。